

SSE 流式协议源码精讲

基于 agplatform `runtime/agent_runtime/api/sse.py` 逐行分析 面向:前端转 Agent 工程师 文件: `/Users/moshangyanyu/Documents/百融/产品研发部/agplatform/runtime/agent_runtime/api/sse.py`

目录

1. 文件定位与作用
2. 模块导入与全局对象
3. SSEEventType 事件类型枚举
4. SSEEvent 事件对象
5. SSEStream 流管理器总览
6. 事件优先级设计
7. 背压丢弃策略(精妙)
8. 入队逻辑
9. 语义化发送方法族
10. OUTPUT_DISCARD 撤回机制
11. 终结事件与关闭
12. 通用事件分发
13. 异步迭代器(最难)
14. 响应生成器
15. 与 FastAPI 的集成
16. 与前端的对应关系
17. 整体架构图
18. 学习要点总结
19. 前端代码对照
20. 自己实现最简版本

1. 文件定位与作用

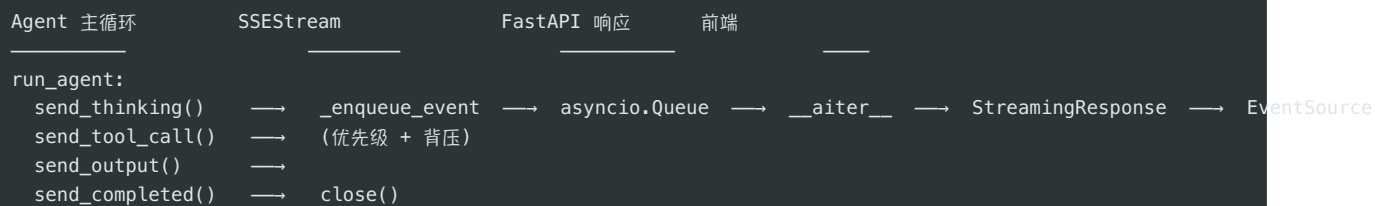
1.1 文件路径

`agplatform/runtime/agent_runtime/api/sse.py`

1.2 作用

这是 agplatform Agent runtime 的 **SSE 流式响应核心**。Agent 执行过程中的所有事件(思考、工具调用、输出、完成)都通过这个模块推送给前端。

1.3 在系统中的位置



前端写 `useSSEChat` 消费的,就是这个文件产生的事件。

1.4 为什么前端工程师要懂这个

- 你前端的 `useSSEChat` 是消费端,这个文件是生产端
- 懂了生产端,你能设计更合理的前端事件处理
- 出 bug 时能判断是前端还是后端问题
- 这是 W5/W6 周学习任务的核心文件

2. 模块导入与全局对象

2.1 代码

```
"""
SSE (Server-Sent Events) Streaming Support
Provides utilities for streaming agent execution events to clients.
"""

import asyncio
import json
import logging
from typing import Any, Optional, AsyncGenerator
from dataclasses import dataclass, field
from datetime import datetime, timezone
from enum import Enum

logger = logging.getLogger(__name__)
```

2.2 导入逐项说明

导入	作用	前端类比
<code>asyncio</code>	Python 异步核心库	Promise / async-await
<code>json</code>	JSON 序列化	JSON.stringify / parse
<code>logging</code>	日志模块	console.log
<code>Any</code>	任意类型	TS 的 <code>any</code>
<code>Optional</code>	可空类型	TS 的 <code>T null</code>
<code>AsyncGenerator</code>	异步生成器类型	AsyncIterator
<code>dataclass</code>	数据类装饰器	TS interface + 自动构造函数
<code>field</code>	dataclass 字段定义	—
<code>datetime, timezone</code>	时间处理	Date
<code>Enum</code>	枚举基类	TS enum

2.3 关键点

```
logger = logging.getLogger(__name__)
```

- `__name__` 是当前模块名(如 `agentic_runtime.api.sse`)
- 创建本模块专属 logger,日志可按模块过滤
- 类比前端: `const logger = createLogger('sse')`

3. SSEEventType 事件类型枚举

3.1 代码

```
class SSEEventType(str, Enum):
    """Types of SSE events"""
    # Connection events
    CONNECTED = "connected"
    DISCONNECTED = "disconnected"
    HEARTBEAT = "heartbeat"

    # Execution events
    STARTED = "started"
    THINKING = "thinking"
    TOOL_CALL = "tool_call"
    TOOL_RESULT = "tool_result"
    ARTIFACT = "artifact"
    ACTION = "action"
    UI_ACTION = "ui_action"
    PROGRESS = "progress"
    OUTPUT = "output"
    OUTPUT_DISCARD = "output_discard"
    COMPLETED = "completed"
    ERROR = "error"

    # Workflow events
    NODE_STARTED = "node_started"
    NODE_COMPLETED = "node_completed"
```

3.2 继承设计

```
class SSEEventType(str, Enum):
```

继承 `str, Enum`, 让枚举值既是枚举又是字符串:

- 方便 JSON 序列化(直接当字符串用)
- 方便比较(`event.event == "thinking"` 也能工作)

3.3 三类事件详解

连接类事件

事件	含义	前端处理
CONNECTED	连接建立	显示"已连接", 记录 session_id
DISCONNECTED	断开	清理状态
HEARTBEAT	心跳保活	忽略即可(防代理超时)

HEARTBEAT 的关键作用: LLM 推理慢(几秒到几十秒), 如果不发心跳, nginx / 浏览器会超时断连。15 秒心跳保证连接存活。

执行类事件

事件	含义	前端处理
STARTED	执行开始	显示"开始执行"
THINKING	Agent 思考中	显示 loading 动画
TOOL_CALL	要调工具	显示工具卡片(参数、状态)
TOOL_RESULT	工具返回	更新工具卡片结果
ARTIFACT	产出物(如文件)	显示下载链接
ACTION	动作	通用动作展示
UI_ACTION	UI 动作	让前端做点什么(如跳转)
PROGRESS	进度	显示进度条(30%)
OUTPUT	输出文本(流式)	逐字拼接显示
OUTPUT_DISCARD	撤回前言	删除尾部文字
COMPLETED	完成	关闭 loading,结束
ERROR	出错	显示错误,关闭连接

这是 Agent 循环的核心信号:

- THINKING → 前端转圈
- TOOL_CALL → 前端显示工具卡片
- OUTPUT → 前端逐字拼接(rAF 批处理)

工作流事件

```
NODE_STARTED = "node_started"
NODE_COMPLETED = "node_completed"
```

工作流(Workflow)相关,用于多步骤编排场景。普通 Agent 用不到。

3.4 OUTPUT_DISCARD 的精妙设计

```
# Retract pre-tool narration: text streamed before a tool call is preface
# leading up to that call. The frontend drops the matching trailing text.
OUTPUT_DISCARD = "output_discard"
```

场景:LLM 有时会先输出一段文字,然后才调工具。比如:

```
用户:帮我看 /tmp/a.py
Agent 输出:"好的,我来读一下" ← 这是工具调用的前言
Agent 调 FileRead 工具      ← 真正的动作
```

这段"好的,我来读一下"其实是工具调用的前言,不该显示给用户。 OUTPUT_DISCARD 通知前端"把刚才那段文字撤回"。

为什么重要:这是产品体验细节,不做的话用户会看到一堆"好的我来读一下"的废话。

4. SSEEvent 事件对象

4.1 代码

```
@dataclass
class SSEEvent:
    """Represents a single SSE event"""
    event: SSEEventType
    data: Any
    id: Optional[str] = None
    retry: Optional[int] = None
    timestamp: str = field(default_factory=lambda: datetime.now(timezone.utc).isoformat())
```

4.2 @dataclass 装饰器

@dataclass 自动生成:

- `__init__` (构造函数)
- `__repr__` (打印)
- `__eq__` (比较)

等价于手写:

```
class SSEEvent:
    def __init__(self, event, data, id=None, retry=None, timestamp=None):
        self.event = event
        self.data = data
        self.id = id
        self.retry = retry
        self.timestamp = timestamp or datetime.now(timezone.utc).isoformat()
```

4.3 字段说明

字段	类型	作用
<code>event</code>	SSEEventType	事件类型
<code>data</code>	Any	事件数据(任意类型)
<code>id</code>	Optional[str]	事件 ID(断线重连用)
<code>retry</code>	Optional[int]	重连等待毫秒
<code>timestamp</code>	str	创建时间(自动)

4.4 field(default_factory=...) 的玄机

```
timestamp: str = field(default_factory=lambda: datetime.now(timezone.utc).isoformat())
```

为什么不用 `default=...` ?

```
# 错误写法 (所有实例共享同一个时间)
timestamp: str = datetime.now(timezone.utc).isoformat()

# 正确写法 (每次创建实例都调一次)
timestamp: str = field(default_factory=lambda: datetime.now(timezone.utc).isoformat())
```

`default=` 是类加载时计算一次,所有实例共享。 `default_factory=` 是每次实例化都调用,保证每个事件有独立时间戳。

4.5 to_sse_string 方法

```
def to_sse_string(self) -> str:
    """Convert to SSE format string"""
    lines = []

    if self.id:
        lines.append(f"id: {self.id}")

    if self.retry:
        lines.append(f"retry: {self.retry}")

    lines.append(f"event: {self.event.value}")

    # Serialize data
    if isinstance(self.data, dict):
        payload = dict(self.data)
        payload["timestamp"] = self.timestamp
        data_str = json.dumps(payload, ensure_ascii=False)
    elif isinstance(self.data, str):
        data_str = json.dumps({"message": self.data, "timestamp": self.timestamp})
    else:
        data_str = json.dumps({"value": self.data, "timestamp": self.timestamp})

    # SSE data can span multiple lines
    for line in data_str.split("\n"):
        lines.append(f"data: {line}")

    lines.append("") # Empty line to end the event
    return "\n".join(lines) + "\n"
```

4.6 SSE 协议格式

SSE 协议规定:

```
id: 123          ← 可选,事件 ID
event: thinking  ← 事件类型
data: {"text": "分析中"} ← 数据(可多行)
```

关键:事件之间用空行分隔。

4.7 序列化三种情况

```
if isinstance(self.data, dict):
    payload = dict(self.data)
    payload["timestamp"] = self.timestamp
    data_str = json.dumps(payload, ensure_ascii=False)
elif isinstance(self.data, str):
    data_str = json.dumps({"message": self.data, "timestamp": self.timestamp})
else:
    data_str = json.dumps({"value": self.data, "timestamp": self.timestamp})
```

输入类型	输出格式
dict	直接加 timestamp
str	包成 <code>{"message": ...}</code>
其他(数字、list)	包成 <code>{"value": ...}</code>

4.8 ensure_ascii=False

```
data_str = json.dumps(payload, ensure_ascii=False)
```

- `ensure_ascii=True` (默认):中文转义成 `\u4f60\u597d`
- `ensure_ascii=False` :中文原样输出"你好"

为什么重要:转义后前端能看到中文,但调试时难读。原样输出方便排查问题。

4.9 多行 data 处理

```
for line in data_str.split("\n"):
    lines.append(f"data: {line}")
```

SSE 协议:data 字段如果跨行,每行都要加 `data:` 前缀。虽然 JSON 通常没换行,但这是防御性编程。

5. SSEStream 流管理器总览

5.1 类定义

```
class SSEStream:
    """
    Manages an SSE stream with event buffering and async iteration.
    """
```

这是整个文件的核心。Agent 主循环往里写事件,FastAPI 响应从里面读事件。

5.2 构造函数

```
def __init__(self, heartbeat_interval: int = 15, max_queue_size: int = 128):
    self._queue: asyncio.Queue[SSEEvent] = asyncio.Queue(maxsize=max_queue_size)
    self._closed = False
    self._closed_event = asyncio.Event()
    self._heartbeat_interval = heartbeat_interval
    self._max_queue_size = max_queue_size
    self._event_id = 0
    self._dropped_events = 0
```

5.3 字段详解

字段	类型	作用
<code>_queue</code>	<code>asyncio.Queue</code>	有界队列(128),生产者写,消费者读
<code>_closed</code>	<code>bool</code>	是否关闭
<code>_closed_event</code>	<code>asyncio.Event</code>	异步事件,通知迭代器退出
<code>heartbeat_interval</code>	<code>int</code>	心跳间隔(15 秒)
<code>_max_queue_size</code>	<code>int</code>	队列最大容量
<code>_event_id</code>	<code>int</code>	自增 ID
<code>_dropped_events</code>	<code>int</code>	丢弃事件计数(监控)

5.4 前端类比

Python 概念	前端类比
<code>asyncio.Queue</code>	消息队列(自己实现)
<code>asyncio.Event</code>	Promise + resolve
有界队列	背压控制

5.5 为什么队列大小 128

- 太小:容易满,频繁触发丢弃
- 太大:内存占用高,延迟积累
- 128 是经验值,平衡点

6. 事件优先级设计

6.1 代码

```
@staticmethod
def _event_priority(event: SSEEvent) -> int:
    if event.event == SSEEventType.ERROR:
        return 4
    if event.event in {SSEEventType.ACTION, SSEEventType.UI_ACTION}:
        return 3
    if event.event in {
        SSEEventType.STARTED,
        SSEEventType.TOOL_CALL,
        SSEEventType.TOOL_RESULT,
        SSEEventType.COMPLETED,
        # Must not be dropped while its paired TOOL_CALL survives, or the
        # pre-tool narration it retracts would stay visible on the frontend.
        SSEEventType.OUTPUT_DISCARD,
    }:
        return 2
    return 1
```

6.2 优先级表

优先级	事件	为什么
4(最高)	ERROR	错误必须送达,否则用户不知道出错了
3	ACTION, UI_ACTION	用户交互动作,丢了影响体验
2	STARTED, TOOL_CALL, TOOL_RESULT, COMPLETED, OUTPUT_DISCARD	执行主线,必须保
1(最低)	THINKING, OUTPUT, PROGRESS, HEARTBEAT 等	可丢(高频,丢几个无所谓)

6.3 OUTPUT_DISCARD 为什么是 2 不是 1

看注释:

```
# Must not be dropped while its paired TOOL_CALL survives, or the
# pre-tool narration it retracts would stay visible on the frontend.
SSEEventType.OUTPUT_DISCARD,
```

场景:

1. LLM 输出 "好的我来读一下"(OUTPUT)
2. LLM 调 FileRead(TOOL_CALL)
3. 系统发 OUTPUT_DISCARD 撤回那段文字

如果 **TOOL_CALL** 存活(优先级 2),但 **OUTPUT_DISCARD** 被丢(如果它是优先级 1),前端就会一直显示"好的我来读一下"。

所以 **OUTPUT_DISCARD** 必须和 **TOOL_CALL** 同优先级,成对存活。

6.4 设计要点

- ERROR 最高:保证用户看到错误
 - THINKING 最低:思考状态丢几个无所谓
 - 高频事件(OUTPUT)低优先级:流式文字丢几个字影响小
-

7. 背压丢弃策略(精妙)

7.1 问题

队列满了怎么办?

- 阻塞:Agent 主循环卡住,不可接受
- 丢最新:可能丢重要事件
- 丢最旧:可能丢未处理的事件

7.2 解决方案:按优先级丢

```
def _drop_stale_event_for(self, incoming: SSEEvent) -> bool:
    """
    Free one queue slot without sacrificing higher-priority actions.

    Returns False when the incoming event should be dropped instead of
    evicting queued events with higher delivery priority.
    """
    incoming_priority = self._event_priority(incoming)
    buffered: list[SSEEvent] = []

    while True:
        try:
            buffered.append(self._queue.get_nowait())
        except asyncio.QueueEmpty:
            break

    if not buffered:
        return True
```

7.3 逻辑步骤

1. 取出队列所有事件到 `buffered`
2. 队列空 → 直接放(返回 True)

```

drop_index: int | None = None
drop_priority = incoming_priority + 1
for index, current in enumerate(buffered):
    current_priority = self._event_priority(current)
    if (
        current_priority <= incoming_priority
        and current_priority < drop_priority
    ):
        drop_index = index
        drop_priority = current_priority

```

1. 遍历找优先级 ≤ 进来事件且最低的那个

```

if drop_index is not None:
    del buffered[drop_index]
    self._dropped_events += 1
    freed = True
else:
    freed = False

for current in buffered:
    self._queue.put_nowait(current)

return freed

```

1. 找到就删,计数 +1
2. 把剩下的放回队列
3. 返回是否腾出位置

7.4 边界情况

如果队列里全是高优先级事件,进来的低优先级事件怎么办?

- `drop_index = None` (找不到能丢的)
- `freed = False`
- 进来的事件被丢弃(在 `_enqueue_event` 里处理)

设计哲学:宁可丢新来的低优先级,也不丢队列里的高优先级。

8. 入队逻辑

8.1 代码

```
def _enqueue_event(self, event: SSEEvent) -> None:
    """Insert an event without blocking. Preserve high-priority events."""
    while True:
        try:
            self._queue.put_nowait(event)
            if self._dropped_events and self._dropped_events % 50 == 0:
                logger.warning(
                    "SSE queue overflow: dropped %s events",
                    self._dropped_events,
                )
            return
        except asyncio.QueueFull:
            if not self._drop_stale_event_for(event):
                self._dropped_events += 1
                if self._dropped_events % 50 == 0:
                    logger.warning(
                        "SSE queue overflow: dropped %s events",
                        self._dropped_events,
                    )
            return
```

8.2 流程

1. 尝试无阻塞入队(`put_nowait`)
2. 成功 → 检查丢弃计数,每 50 个警告一次
3. 队列满 → 调 `_drop_stale_event_for` 腾位置
4. 腾出位置 → 重试入队(while True)
5. 腾不出 → 丢弃当前事件,计数 +1

8.3 为什么每 50 个警告一次

```
if self._dropped_events and self._dropped_events % 50 == 0:
    logger.warning(...)
```

- 每丢一个都警告 → 日志洪水
- 不警告 → 问题发现不了
- 每 50 个一次 → 平衡

8.4 while True 不会死循环吗

不会。因为:

- 队列有界(128)
- `_drop_stale_event_for` 要么腾位置,要么返回 False
- 返回 False 时直接 return(丢弃当前事件)

9. 语义化发送方法族

9.1 通用发送方法

```
async def send(
    self,
    event_type: SSEEventType,
    data: Any,
    event_id: Optional[str] = None
) -> None:
    """Send an event to the stream"""
    if self._closed:
        return

    self._event_id += 1
    event = SSEEvent(
        event=event_type,
        data=data,
        id=event_id or str(self._event_id)
    )
    self._enqueue_event(event)
```

9.2 要点

- `if self._closed: return` :关闭后不发(防止资源浪费)
- `self._event_id += 1` :自增 ID
- `event_id or str(self._event_id)` :优先用传入的 ID

9.3 语义化方法族

每个事件类型一个方法,代码可读性高:

```

async def send_connected(self, session_id: str) -> None:
    await self.send(SSEEventType.CONNECTED, {
        "session_id": session_id,
        "message": "SSE connection established"
    })

async def send_started(self, agent_id: str, request_id: str) -> None:
    await self.send(SSEEventType.STARTED, {
        "agent_id": agent_id,
        "request_id": request_id,
        "message": "Agent execution started"
    })

async def send_thinking(self, thought: str) -> None:
    await self.send(SSEEventType.THINKING, {"text": thought})

async def send_tool_call(
    self, tool_name: str, tool_input: dict, call_id: Optional[str] = None
) -> None:
    await self.send(SSEEventType.TOOL_CALL, {
        "tool_name": tool_name,
        "tool_input": tool_input,
        "call_id": call_id
    })

async def send_tool_result(
    self, tool_name: str, result: Any, success: bool,
    error: Optional[str] = None, duration_ms: int = 0
) -> None:
    await self.send(SSEEventType.TOOL_RESULT, {
        "tool_name": tool_name,
        "result": result,
        "success": success,
        "error": error,
        "duration_ms": duration_ms
    })

async def send_progress(
    self, progress: int, message: Optional[str] = None,
    current_step: Optional[int] = None, total_steps: Optional[int] = None
) -> None:
    await self.send(SSEEventType.PROGRESS, {
        "progress": progress,
        "message": message,
        "current_step": current_step,
        "total_steps": total_steps
    })

async def send_output(self, output: Any, partial: bool = False) -> None:
    await self.send(SSEEventType.OUTPUT, {
        "output": output,
        "partial": partial
    })

```

9.4 方法对照表

方法	事件	关键字段	前端用途
<code>send_connected</code>	CONNECTED	session_id	记录会话
<code>send_started</code>	STARTED	agent_id, request_id	显示开始
<code>send_thinking</code>	THINKING	text	loading 动画
<code>send_tool_call</code>	TOOL_CALL	tool_name, tool_input, call_id	工具卡片
<code>send_tool_result</code>	TOOL_RESULT	result, success, duration_ms	卡片更新
<code>send_progress</code>	PROGRESS	progress, current_step	进度条
<code>send_output</code>	OUTPUT	output, partial	逐字拼接

9.5 call_id 的作用

```
async def send_tool_call(self, tool_name, tool_input, call_id=None):
    await self.send(SSEEventType.TOOL_CALL, {
        "tool_name": tool_name,
        "tool_input": tool_input,
        "call_id": call_id      # 关键
    })
```

`call_id` 用于匹配 `tool_call` 和 `tool_result`。Agent 可能同时调多个工具,前端要靠 `call_id` 知道哪个结果对应哪个调用。

9.6 duration_ms 的用途

```
async def send_tool_result(self, ..., duration_ms: int = 0):
```

前端显示工具耗时(你做过),让用户知道工具执行了多久。

9.7 partial 字段

```
async def send_output(self, output: Any, partial: bool = False):
```

- `partial=True` :部分输出(流式中)
- `partial=False` :完整输出

前端可根据这个决定是否立即渲染还是缓冲。

10. OUTPUT_DISCARD 撤回机制

10.1 代码

```
async def send_output_discard(self, discarded: str) -> None:
    """Retract previously streamed pre-tool narration text.

    Emitted right before a ``tool_call`` event when the text streamed
    since the last boundary (turn start or previous tool call) turned
    out to be a tool-invocation preface. The frontend removes the
    matching trailing text from the current assistant message.

    Args:
        discarded: The exact preface text that was streamed and should
            now be removed (frontend strips this suffix / ``chars`` count).
    """
    await self.send(SSEEventType.OUTPUT_DISCARD, {
        "discarded": discarded,
        "chars": len(discarded),
    })
```

10.2 场景

1. LLM 输出:"好的,我来读一下" (OUTPUT 事件)
2. 前端显示:"好的,我来读一下"
3. LLM 决定调 FileRead (TOOL_CALL 事件)
4. 系统发 OUTPUT_DISCARD (撤回那段文字)
5. 前端删除:"好的,我来读一下"
6. 前端显示工具卡片

10.3 chars 字段的优化

```
"chars": len(discarded),
```

前端可以:

- 按 `discarded` 字符串匹配删除(不稳定,如果文本有微小差异)
- 按 `chars` 字符数删除尾部(更稳定)

推荐用 `chars` ,因为字符串匹配可能因为空白字符等问题。

10.4 触发时机

注释说: `Emitted right before a tool_call event`

顺序固定:

11. 终结事件与关闭

11.1 代码

```
async def send_completed(
    self, success: bool, output: Any = None,
    duration_ms: int = 0, iterations: int = 0
) -> None:
    """Send execution completed event"""
    await self.send(SSEEventType.COMPLETED, {
        "success": success,
        "output": output,
        "duration_ms": duration_ms,
        "iterations": iterations
    })
    self.close()    # 发完就关流

async def send_error(self, error: str, code: Optional[str] = None) -> None:
    """Send error event"""
    await self.send(SSEEventType.ERROR, {
        "error": error,
        "code": code
    })
    self.close()    # 出错也关流
```

11.2 字段说明

字段	含义
<code>success</code>	是否成功
<code>output</code>	最终输出
<code>duration_ms</code>	总耗时
<code>iterations</code>	Agent 循环了几轮(工具调用次数)
<code>code</code>	错误码(如 RATE_LIMIT / TIMEOUT)

11.3 iterations 的用途

前端可显示"Agent 思考了 N 步",让用户感知 Agent 的工作量。

11.4 self.close() 的重要性

```
self.close()
```

发完终结事件立即关闭,防止后续事件漏发。`close()` 会:

- 设置 `_closed = True`
- 设置 `_closed_event` (唤醒迭代器)

11.5 心跳方法

```
async def send_heartbeat(self) -> None:
    """Send heartbeat to keep connection alive"""
    if not self._closed:
        await self.send(SSEEventType.HEARTBEAT, {
            "message": "ping"
        })
```

心跳内容很简单,就是个 `{"message": "ping"}`。

12. 通用事件分发

12.1 代码

```
async def send_event(self, event: str, data: Any) -> None:
    """
    Send a generic event with custom event type.

    This is a convenience method for sending events that don't have
    a dedicated send_* method.
    """
    event_type_map = {
        "connected": SSEEventType.CONNECTED,
        "started": SSEEventType.STARTED,
        "thinking": SSEEventType.THINKING,
        "tool_call": SSEEventType.TOOL_CALL,
        "tool_result": SSEEventType.TOOL_RESULT,
        "artifact": SSEEventType.ARTIFACT,
        "action": SSEEventType.ACTION,
        "ui_action": SSEEventType.UI_ACTION,
        "progress": SSEEventType.PROGRESS,
        "output": SSEEventType.OUTPUT,
        "output_discard": SSEEventType.OUTPUT_DISCARD,
        "completed": SSEEventType.COMPLETED,
        "error": SSEEventType.ERROR,
        "heartbeat": SSEEventType.HEARTBEAT,
    }
    event_type = event_type_map.get(event)
    if event_type:
        await self.send(event_type, data)
    else:
        await self.send(SSEEventType.OUTPUT, {"event": event, **data})
```

12.2 设计意图

允许用字符串调(方便动态调用),未知事件降级成 OUTPUT。

12.3 向前兼容

```
else:
    await self.send(SSEventType.OUTPUT, {"event": event, **data})
```

以后加新事件类型,旧代码不会崩,会降级成 OUTPUT。这是防御性编程。

13. 异步迭代器(最难)

13.1 代码

```
async def __aiter__(self) -> AsyncGenerator[str, None]:
    """Async iterator for streaming events"""
    pending_get: Optional[asyncio.Task] = None
    try:
        while True:
            if self._closed and self._queue.empty():
                break

            if pending_get is None:
                pending_get = asyncio.create_task(self._queue.get())

            close_wait = asyncio.create_task(self._closed_event.wait())
            try:
                done, _ = await asyncio.wait(
                    {pending_get, close_wait},
                    timeout=self._heartbeat_interval,
                    return_when=asyncio.FIRST_COMPLETED,
                )
                if pending_get in done:
                    event = pending_get.result()
                    pending_get = None
                    yield event.to_sse_string()
                    continue

                if self._closed and self._queue.empty():
                    break

                if close_wait in done and self._queue.empty():
                    break

                if not self._closed:
                    await self.send_heartbeat()
            finally:
                if not close_wait.done():
                    close_wait.cancel()
        except asyncio.CancelledError:
            # Client disconnected
            self._closed = True
            self._closed_event.set()
            raise
    finally:
        if pending_get is not None and not pending_get.done():
            pending_get.cancel()
```

13.2 aiter 的作用

让对象可以被 `async for` 遍历:

```
async for event_str in stream:
    yield event_str
```

13.3 退出条件

```
if self._closed and self._queue.empty():
    break
```

已关闭且队列空才退出。即使关闭了,队列里还有事件也要发完。

13.4 双任务并行等待

```
pending_get = asyncio.create_task(self._queue.get())
close_wait = asyncio.create_task(self._closed_event.wait())

done, _ = await asyncio.wait(
    {pending_get, close_wait},
    timeout=self._heartbeat_interval,
    return_when=asyncio.FIRST_COMPLETED,
)
```

同时等两件事:

1. 队列有事件(`pending_get`)
2. 收到关闭信号(`close_wait`)

带 15 秒超时,谁先完成谁触发。

13.5 事件处理

```
if pending_get in done:
    event = pending_get.result()
    pending_get = None
    yield event.to_sse_string()
    continue
```

取到事件 → yield 出去(给 StreamingResponse)→ continue 继续。

13.6 心跳触发

```
if not self._closed:
    await self.send_heartbeat()
```

15 秒超时没事件 → 发心跳保活。

13.7 资源清理

```
finally:
    if not close_wait.done():
        close_wait.cancel()
```

每次循环结束取消未完成的 `close_wait`, 避免任务泄漏。

13.8 客户端断开处理

```
except asyncio.CancelledError:
    # Client disconnected
    self._closed = True
    self._closed_event.set()
    raise
```

客户端断开(SSE 连接断)→ FastAPI 抛 `CancelledError` → 标记关闭。

13.9 最终清理

```
finally:
    if pending_get is not None and not pending_get.done():
        pending_get.cancel()
```

退出前取消 pending task, 防内存泄漏。

14. 响应生成器

14.1 代码

```
async def create_sse_response(
    stream: SSEStream,
) -> AsyncGenerator[str, None]:
    """
    Create an SSE response generator from a stream.
    Use with FastAPI's StreamingResponse.
    """
    async for event_str in stream:
        yield event_str
```

14.2 作用

简单包装, 方便 FastAPI 使用。

15. 与 FastAPI 的集成

15.1 使用示例

```
from fastapi import FastAPI
from fastapi.responses import StreamingResponse
import asyncio

app = FastAPI()

@app.get("/chat/stream")
async def chat_stream():
    stream = SSEStream()
    # 后台跑 Agent, 往 stream 里写事件
    asyncio.create_task(run_agent(stream))
    # 返回 SSE 响应
    return StreamingResponse(
        create_sse_response(stream),
        media_type="text/event-stream"
    )

async def run_agent(stream: SSEStream):
    await stream.send_connected("session-123")
    await stream.send_thinking("分析用户问题...")
    await asyncio.sleep(2) # 模拟 LLM 推理
    await stream.send_output("你好,世界!", partial=False)
    await stream.send_completed(success=True, output="你好,世界!")
```

15.2 关键点

- `asyncio.create_task(run_agent(stream))` : 后台并发跑 Agent
 - `StreamingResponse` : FastAPI 流式响应
 - `media_type="text/event-stream"` : SSE MIME 类型
-

16. 与前端的对应关系

16.1 事件映射

后端事件	前端处理(你写过的)
CONNECTED	<code>setConnected(true)</code> ,记录 session_id
THINKING	<code>setThinking(true)</code>
OUTPUT	<code>setMessage(prev + delta)</code> (raf 批处理)
TOOL_CALL	<code>setTools(prev => [...prev, tool])</code>
TOOL_RESULT	更新对应 tool 的 result
COMPLETED	<code>setThinking(false)</code> ,关闭连接
ERROR	显示错误,关闭连接
HEARTBEAT	忽略(保活用)

16.2 前端伪代码

```
const eventSource = new EventSource('/chat/stream');

eventSource.addEventListener('connected', (e) => {
  const { session_id } = JSON.parse(e.data);
  setSessionId(session_id);
});

eventSource.addEventListener('thinking', (e) => {
  setThinking(true);
});

eventSource.addEventListener('tool_call', (e) => {
  const tool = JSON.parse(e.data);
  setTools(prev => [...prev, { ...tool, status: 'pending' }]);
});

eventSource.addEventListener('tool_result', (e) => {
  const result = JSON.parse(e.data);
  setTools(prev => prev.map(t =>
    t.call_id === result.call_id
      ? { ...t, status: 'success', result: result.result }
      : t
  ));
});

eventSource.addEventListener('output', (e) => {
  const { output, partial } = JSON.parse(e.data);
  // rAF 批处理
  requestAnimationFrame(() => {
    setMessage(prev => prev + output);
  });
});

eventSource.addEventListener('completed', (e) => {
  setThinking(false);
  eventSource.close();
});
```

17. 整体架构图

Agent 主循环 (run_agent)

```
await stream.send_thinking("分析中")
await stream.send_tool_call("FileRead", {path: "/tmp/a.py"})
await stream.send_tool_result("FileRead", content)
await stream.send_output("这是结果", partial=True)
await stream.send_completed(success=True)
```

SSEStream

_enqueue_event

- └ put_nowait (尝试入队)
- └ 队列满 → _drop_stale_event_for
 - └ 按优先级丢低优先级
- └ 入队成功

asyncio.Queue (maxsize=128)
[event1, event2, event3, ...]

__aiter__ (异步迭代器)

- └ asyncio.wait(queue.get, close_event)
- └ timeout=15s → send_heartbeat
- └ yield event.to_sse_string()

FastAPI StreamingResponse
media_type="text/event-stream"

浏览器 EventSource

```
eventSource.addEventListener('thinking', ...)
eventSource.addEventListener('tool_call', ...)
eventSource.addEventListener('output', ...)
```

18. 学习要点总结

18.1 核心设计点

1. 事件类型设计:连接 / 执行 / 工作流三类,前端按需监听
2. 背压策略:队列满时按优先级丢低优先级事件,保高优先级
3. 心跳保活:15 秒无事件发心跳,防代理超时
4. OUTPUT_DISCARD:撤回工具调用前的前言文字(产品体验细节)
5. 异步迭代器:用 `asyncio.wait` 同时等"取事件"和"关闭信号",带超时
6. 资源清理:客户端断开时取消所有 pending task

18.2 设计模式

模式	应用
策略模式	<code>_event_priority</code> 按事件类型算优先级
模板方法	<code>to_sse_string</code> 的序列化流程
观察者	<code>asyncio.Event</code> 通知迭代器退出
背压	队列满时丢弃策略

18.3 关键技术

技术	作用
<code>asyncio.Queue</code>	有界队列,生产者消费者解耦
<code>asyncio.Event</code>	异步信号
<code>asyncio.wait</code>	多任务并行等待
<code>@dataclass</code>	数据类自动生成
<code>Enum</code>	枚举类型安全
<code>AsyncGenerator</code>	异步迭代器

19. 前端代码对照

19.1 你在 autobots 写过的

你在 autobots 前端写过 SSE 消费代码,现在对照后端:

19.2 事件对照

后端发送	前端接收
<code>stream.send_connected(session_id)</code>	<code>eventSource.addEventListener('connected', ...)</code>
<code>stream.send_thinking(text)</code>	<code>addEventListener('thinking', ...)</code>
<code>stream.send_tool_call(...)</code>	<code>addEventListener('tool_call', ...)</code>
<code>stream.send_output(delta)</code>	<code>addEventListener('output', ...)</code> → 拼接文字
<code>stream.send_completed()</code>	<code>addEventListener('completed', ...)</code> → 关闭
<code>stream.send_error(msg)</code>	<code>addEventListener('error', ...)</code> → 显示错误

19.3 你前端的 rAF 批处理

后端一秒能吐几十个 OUTPUT 事件,直接 setState 会卡。你用 rAF 批量 flush:

```
let pendingText = '';
let rafId: number;

eventSource.addEventListener('output', (e) => {
  const { output } = JSON.parse(e.data);
  pendingText += output;
  if (!rafId) {
    rafId = requestAnimationFrame(() => {
      setMessage(prev => prev + pendingText);
      pendingText = '';
      rafId = 0;
    });
  }
});
```

这是前端标准优化,后端的 128 队列 + 背压是后端对应的优化。

20. 自己实现最简版本

20.1 最简 SSEStream(学习用)

```
import asyncio
import json
from datetime import datetime, timezone

class SimpleSSEStream:
    def __init__(self):
        self._queue = asyncio.Queue(maxsize=64)
        self._closed = False

    async def send(self, event: str, data: dict):
        if self._closed:
            return
        payload = {
            "event": event,
            "data": data,
            "timestamp": datetime.now(timezone.utc).isoformat()
        }
        await self._queue.put(payload) # 注意:这里会阻塞,生产用 put_nowait

    async def send_output(self, text: str):
        await self.send("output", {"text": text})

    async def send_completed(self):
        await self.send("completed", {})
        self._closed = True

    async def __aiter__(self):
        while True:
            if self._closed and self._queue.empty():
                break
            try:
                event = await asyncio.wait_for(self._queue.get(), timeout=15)
                yield self._format(event)
            except asyncio.TimeoutError:
                yield self._format({"event": "heartbeat", "data": {}})

    def _format(self, event: dict) -> str:
        return f"event: {event['event']}\ndata: {json.dumps(event['data'], ensure_ascii=False)}\n\n"
```

20.2 配合 FastAPI

```
from fastapi import FastAPI
from fastapi.responses import StreamingResponse

app = FastAPI()

@app.get("/stream")
async def stream():
    sse = SimpleSSEStream()
    asyncio.create_task(run_agent(sse))
    return StreamingResponse(sse, media_type="text/event-stream")

async def run_agent(sse: SimpleSSEStream):
    await sse.send_output("你好")
    await asyncio.sleep(1)
    await sse.send_output("世界")
    await sse.send_completed()
```

20.3 学习任务

- [] 把这个最简版本跑起来
- [] 加上背压丢弃(参考源码 `_drop_stale_event_for`)
- [] 加上 OUTPUT_DISCARD 逻辑
- [] 加上心跳
- [] 对比和源码的差异,理解为什么源码要那样设计

结语

这个文件是 agplatform Agent runtime 的 SSE 核心,也是你前端 SSE 消费代码的"生产端"。

读懂这个文件,你能:

1. 理解 Agent 执行过程中的事件流
2. 设计更合理的前端事件处理
3. 出 bug 时判断是前端还是后端问题
4. 自己写一个简化版的 SSE Agent 后端

这是 W5/W6 周学习任务的核心文件,建议反复读 3-5 遍,每读一遍都会有新收获。

本文档基于 agplatform `runtime/agent_runtime/api/sse.py` 源码分析 生成时间:2026-08-20